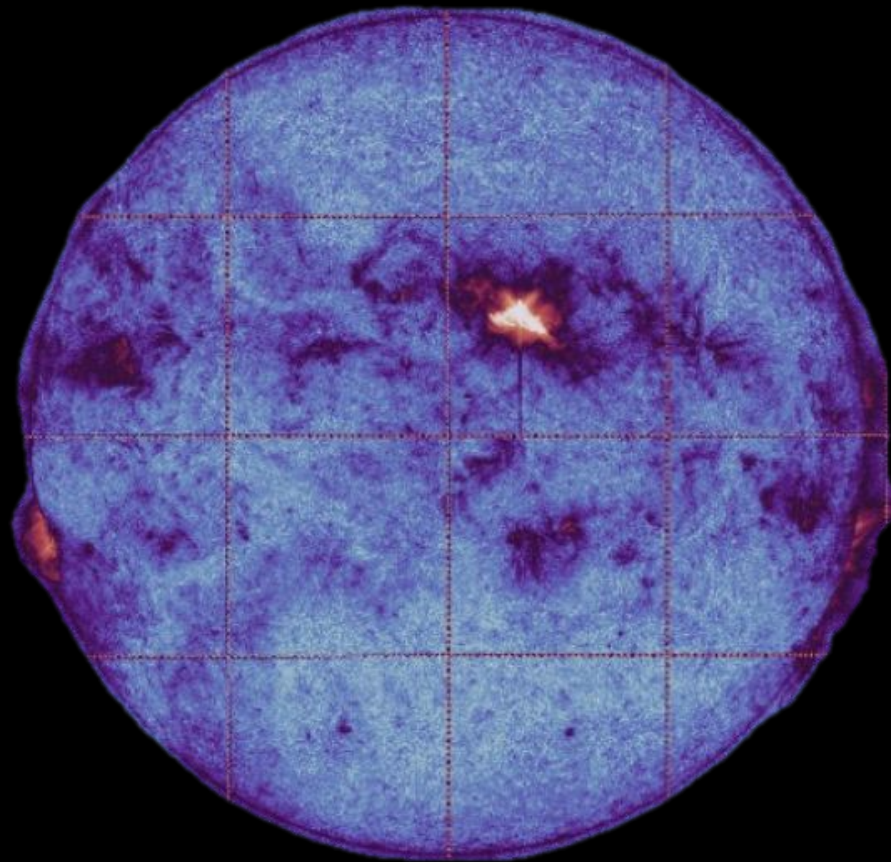


Solar Flare Detection

Machine Learning Classification on SDOBenchmark Dataset



Project Overview & Motivation

Why This Matters

Solar flares damage satellites, GPS, power grids. Early warning systems can **prevent billions in losses**.

Our Dataset

SDO
Benchmark(Balanced):
4,050 balanced training
images, 1,360 test.
Binary task: M/X flares
vs Others.

The Challenge

Class imbalance: 95%
non-flares, 5% flares. Limited
labeled data. Need high recall
for safety. And identifying the
flares has proven to be a
challenging task as the SOTA sill
stands at Acc 75.5 and F1
0.5639.

Our Solution

Built and compared FOUR
different ML algorithms.
Analyzed balanced vs
unbalanced dataset
impact.

Dataset

- Dataset Name: SDOBenchmark
- Source: Solar Dynamics Observatory (SDO) satellite instruments AIA and HMI
- Data Type: **Magnetogram images** capturing solar active regions
- Image Details:
 - 256×256 pixel JPEG images
 - 10 images per sample over 4 time steps
 - From 2 detectors: 8 from AIA, 2 from HMI
- Sample Size:
 - 8,336 training samples
 - 886 test samples
- Task: Predict solar flare class (especially M and X classes) within 24-hour horizon
- **Challenge:**
 - Strong class imbalance (approx. 95% non-flare, 5% flare samples)
 - Data gaps and instrumentation decay over time
 - Data non-stationary due to solar cycle variations

Tech Stack at a Glance

KNN (scikit-learn)

- **Core:** `KNeighborsClassifier()` using Euclidean distance, $k=5$, distance weighting.
- **Process:** `.fit()` stores data → `.predict()` finds nearest neighbors → majority vote.

Random Forest (scikit-learn)

- **Core:** `RandomForestClassifier(n_estimators=100, criterion='gini')`.
- **Process:** Each tree trained on bootstrap samples → majority vote → `.feature_importances_` for Gini ranking.

ResNet18 (PyTorch + torchvision)

- **Core:** 18 residual layers with skip connections
 $y=F(x)+xy=F(x)+xy=F(x)+x$.
- **Modifications:** Input conv→1 ch grayscale, FC→binary Sigmoid.
- **Training:** `BCELoss`, `Adam`, backprop through residuals.

Simple CNN (PyTorch)

- **Layers:** `Conv2d` → `ReLU` → `Pool` → `Conv2d` → `ReLU` → `Pool` → `Linear` → `Sigmoid`.
- **Training:** `BCELoss` + `Adam(lr=0.001)`, forward-backward-update loop.

Comprehensive Results: All Models Compared

Model	Accuracy (%)	Recall (%)	F1 Score
ResNet-50 (SOTA)	75.5	56.39	49.67
SimpleBinaryCNN	73.75	80.00	0.75
ResNet-18	73.82	76.00	0.74
Ensemble (SOTA)	—	49.67	0.56
KNN (Balanced)	55.29	18.00	0.28
KNN (Unbalanced)	80.64	~1.00	~0.02
Random Forest (Balanced)	67.00	60.00	—
Random Forest (Unbalanced)	81.00	100.00	0.06

*The SOTA models mentioned here are based on CNN

Key Findings: Deeper Insights

Class Imbalance Crisis

KNN unbalanced: 80% accuracy but 1% recall. Model optimizes for majority class, fails to detect flares.

Data Quality > Quantity

4K balanced samples outperform 32K unbalanced. 50-50 split more important than volume.

Simplicity Often Wins

Simple CNN: 2 layers, 2K parameters vs SOTA 50 layers, 25M parameters. Only 1.75% performance gap.

Recall > Accuracy

Simple CNN recall: 80% vs published ensemble: 49.67%. 2× better flare detection for early warning.

Competitive Despite Lower Complexity

ResNet18: 73.82% from scratch vs SOTA 75.5% with pre-training. Only 1.68% gap and **beats recall and F1 score by 33 and 60 percent respectively on CNN based models**

Thank you